



APPLIED DATA SCIENCE CAPSTONE

Predicting Falcon 9 First-Stage Landing Success

A data science analysis of SpaceX launch outcomes — from data collection through predictive modeling

Abdou A. Diakite

IBM Data Science Professional Certificate

August 11, 2026

Github: <https://github.com/ptah222/IBM-Data-Science-Capstone-Project.git>



OVERVIEW

Executive Summary

- This project aims to predict whether the Falcon 9 first stage will land successfully so that a competing company can make more informed launch bids.
- Methods used: data collection (API + web scraping), data wrangling, exploratory data analysis, interactive visual analytics and predictive classification modeling.
- After comparing multiple models, we find that the Falcon 9 booster has a high probability of successful landing (87.7% based on decision tree classifier). Success is highly correlated with launch site, orbits, boosters and payload mass.
- Success rate has reached 90% in recent years, thanks to new generation of boosters.

87.7%

*Best model accuracy
Decision Tree classifier*

90

*Launch records analyzed
Falcon 9 launches (2010–2020)*

66.7%

*Overall landing success rate
60 of 90 landings succeeded*



BACKGROUND

Introduction

Project Background

- SpaceX advertises Falcon 9 launches at ~\$62M, far below competitors' ~\$165M+, largely because SpaceX reuses the first stage instead of discarding it.
- Reuse is only possible when the first stage successfully lands, so landing success directly determines launch cost.
- A competing company bidding against SpaceX would benefit from estimating this cost — which requires predicting landing success.

Problem Statement

Can we predict whether the Falcon 9 first stage will land successfully, using publicly available launch data — payload mass, orbit type, launch site, booster version, and more?

Key Questions Explored

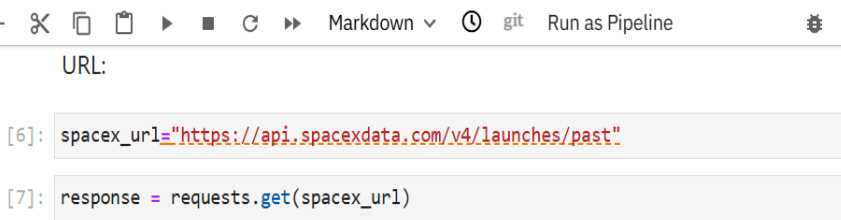
- Which variables most influence landing outcome?
- How has success rate changed over time?
- Which classification model predicts outcome best?



```
graph LR; 1[1 GET request to api.spacexdata.com/v4/launches/past] --> 2[2 Parse JSON response into a pandas DataFrame]; 2 --> 3[3 Use rocket, payload, launchpad, core IDs to make follow-up GET requests for details]; 3 --> 4[4 Merge nested results into flat columns per launch]; 4 --> 5[5 Filter to Falcon 9 launches only (drop Falcon 1)];
```

- 1 GET request to `api.spacexdata.com/v4/launches/past`
- 2 Parse JSON response into a pandas DataFrame
- 3 Use rocket, payload, launchpad, core IDs to make follow-up GET requests for details
- 4 Merge nested results into flat columns per launch
- 5 Filter to Falcon 9 launches only (drop Falcon 1)

- `requests.get(url).json()` against the SpaceX v4 API
- IDs returned per launch (rocket, payloads, launchpad, cores) require separate GET calls to resolve into readable fields
- Custom functions (e.g. `getBoosterVersion()`, `getLaunchSite()`) map IDs → names and append to lists
- Combine into a single DataFrame; export to `spacex_api.csv`



The screenshot shows a JupyterLab environment. The top bar indicates the current file is 'jupyter-labs-spacex-data-ccX'. The toolbar includes icons for saving, adding tabs, undo, redo, running, and other standard JupyterLab actions. The main area contains a code editor with the following Python code:

```
URL:

[6]: spacex_url="https://api.spacexdata.com/v4/launches/past"

[7]: response = requests.get(spacex_url)

Check the content of the response

[8]: print(response.content)
```

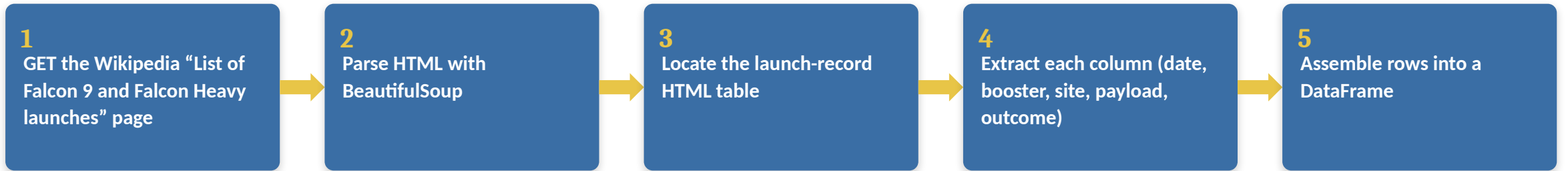
Below the code editor, a terminal window displays the raw HTML response from the API, which is a 525 status error indicating an SSL handshake failure:

```
b'<!DOCTYPE html>\n<!--[if lt IE 7]> <html class="no-js ie6 oldie" lang="en-US"> <![endif]-->\n<!--[if IE 7]> <html class="no-js ie7 oldie" lang="en-US"> <![endif]-->\n<!--[if IE 8]> <html class="no-js ie8 oldie" lang="en-US"> <![endif]-->\n<!--[if gt IE 8]><!--> <html class="no-js" lang="en-US"> <!--<![endif]-->\n<head>\n<title>spacexdata.com | 525: SSL handshake failed</title>\n<meta charset="UTF-8" />\n<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />\n</meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />\n</html>'
```



DATA COLLECTION

Web Scraping (Wikipedia)



Key Steps

- `requests.get()` the Wikipedia revision URL, then BeautifulSoup(html, 'html.parser')
- `soup.find_all('table', 'wikitable plainrowheaders collapsible')` to locate the launch table
- Custom parsing functions per column (date/time, booster version, payload mass, orbit, outcome)
- Append parsed cells into a dict of lists → `pd.DataFrame`; export to `spacex_web_scraped.csv`

```
[11]: # use requests.get() method with the provided static_url and headers
# assign the response to a object
response = requests.get(static_url, headers=headers)

Create a BeautifulSoup object from the HTML response

[12]: soup = BeautifulSoup(response.text, "html.parser")

Print the page title to verify if the BeautifulSoup object was created properly

[13]: print(soup.title)
<title>List of Falcon 9 and Falcon Heavy launches - Wikipedia</title>

TASK 2: Extract all column/variable names from the HTML table header

Next, we want to collect all relevant column names from the HTML table header

Let's try to find all tables on the wiki page first. If you need to refresh your memory about BeautifulSoup, please check the external
reference link towards the end of this lab

[16]: # Use the find_all function in the BeautifulSoup object, with element type 'table'.
html_tables = soup.find_all("table")

Starting from the third table is our target table contains the actual launch records.

[15]: # Let's print the third table and check its content
third_launch_table = html_tables[2]
print(third_launch_table)

<table class="wikitable plainrowheaders collapsible sticky-header" id="2026ytd" style="width: 100%;">
<tbody id="mGGk"><tr id="mGG0"><th id="mGGs" scope="col">Flight No.</th>
<th id="mGGv" scope="col">Date and<br id="mGG0">time (<a href="https://en.wikipedia.org/wiki/Coordinated_Universal_Time" id="mGG4" rel="mw:
Wikilink" title="Coordinated Universal Time">UTC</a></th>
<th id="mGG8" scope="col"><a href="https://en.wikipedia.org/wiki/List_of_Falcon_9_first-stage_boosters" id="mGHA" rel="mw:Wikilink" title="Li
st of Falcon 9 first-stage boosters">Version,<br id="mGHE">booster</a><sup about="mw:2595" class="mw-ref reference" data-mw="{"name":"re
f","attrs":{"name":"booster","group":"lower-alpha"},"body":{"html":""},"parts":[{"template":{"target":{"wt":"efn","href":"."/Template:Efn"},"par
ams":{"name":{"wt":"booster"}},"id":0}}]}</sup></th></tbody></table>
```



Notebook / Code: github.com/ptah222/IBM-Data-Science-Capstone-Project



```
graph LR; A[1. Combine API + scraped datasets; resolve column mismatches] --> B[2. Handle missing values (e.g. impute mean payload mass)]; B --> C[3. Standardize categorical labels (site, orbit, outcome text)]; C --> D[4. Engineer the binary Class label (1 = success, 0 = failure) from landing outcome text]; D --> E[5. Export the clean, model-ready dataset];
```

- 1 Combine API + scraped datasets; resolve column mismatches
- 2 Handle missing values (e.g. impute mean payload mass)
- 3 Standardize categorical labels (site, orbit, outcome text)
- 4 Engineer the binary Class label (1 = success, 0 = failure) from landing outcome text
- 5 Export the clean, model-ready dataset

- Outcome strings like 'True ASDS', 'True RTLS', 'True Ocean' → Class = 1
- 'False ASDS', 'False RTLS', 'None None' → Class = 0
- This binary label becomes the target variable for the classification models

```
labs-jupyter-spacex-Data-wr-X +  
Open in... Python 3 (pykernel)  
TASK 2: Calculate the number and occurrence of each orbit  
Use the method .value_counts() to determine the number and occurrence of each orbit in the column Orbit  
Note: Do not count GTO, as it is a transfer orbit and not itself geostationary.  
[9]: # Apply value_counts on Orbit column  
df['Orbit'].value_counts()  
[9]: Orbit  
GTO 27  
ISS 21  
VLEO 14  
PO 9  
LEO 7  
SSO 5  
MEO 3  
ES-L1 1  
HEO 1  
SO 1  
GEO 1  
Name: count, dtype: int64  
TASK 3: Calculate the number and occurrence of the missions  
Use the method .value_counts() on the column Outcome to determine the number of landing_outcomes. Then assign it to a variable landing_outcomes.  
[13]: # Landing_outcomes = values on Outcome column  
landing_outcomes = df['Outcome'].value_counts()  
landing_outcomes  
[13]: bound method IndexOpsMixin.value_counts of 0 None None  
1 None None  
2 None None  
3 False Ocean  
4 None None  
...  
85 True ASDS  
86 True ASDS  
87 True ASDS  
88 True ASDS
```



EDA with Data Visualization

Charts Used & Purpose

Scatter — Flight Number vs. Launch Site

Reveal whether success improved over successive flights, per site

Scatter — Payload Mass vs. Launch Site

Check whether heavier payloads cluster at certain sites or fail more

Bar — Success Rate by Orbit Type

Compare landing reliability across orbit classes (LEO, GTO, ISS, ...)

Scatter — Flight Number / Payload vs. Orbit Type

Look for interaction effects between orbit, payload, and experience

Line — Yearly Launch Success Trend

Show how success rate evolved as SpaceX iterated on booster design

- Each chart tests one hypothesis about landing success — experience, mass, orbit energy, or time — and every one of them fed a feature into the final classifier.



Notebook / Code: github.com/ptah222/IBM-Data-Science-Capstone-Project



EXPLORATORY ANALYSIS

EDA with SQL

- Loaded the wrangled dataset into a SQLite / Db2 table via %sql / ibm_db
- Queries covered: distinct launch sites, payload totals by customer, average payload by booster, first successful landing date, boosters meeting payload + outcome criteria, mission-outcome counts, max-payload boosters, month-level 2015 records, and a ranked landing-outcome count (2010–2020)
- Each query answers one specific analytical question used later in the results section

Query Categories Covered

Launch Sites

Payloads

Success Rates

Rankings

Time Analysis

```
[16]: %load_ext sql
      The sql extension is already loaded. To reload it, use:
      %reload_ext sql

[17]: import csv, sqlite3
      import prettytable
      prettytable.DEFAULT = 'DEFAULT'

      con = sqlite3.connect("my_data1.db")
      cur = con.cursor()

[18]: !pip install -q pandas

[19]: %sql sqlite:///my_data1.db

[20]: import pandas as pd
      df = pd.read_csv("https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/labs/module_2/data/SpaceX...")
      df.to_sql("SPACEXTBL", con, if_exists='replace', index=False, method="multi")

[20]: 101
```



Notebook / Code: github.com/ptah222/IBM-Data-Science-Capstone-Project



EXPLORATORY ANALYSIS

Interactive Visual Analytics



Folium Map

- Marker for every launch site, colored by success/failure ratio
- Circle markers for each individual launch record, clustered by site
- Distance lines from a launch site to nearby coastline, highway, and city, to explore siting rationale

Why it matters

- The map makes the siting logic visible: every pad is coastal, so a failed first stage falls over open water rather than populated land.



Plotly Dash App

- Dropdown to filter by launch site
- Pie chart of success-rate share across sites (or per-site success vs. failure)
- Range slider + scatter chart of payload mass vs. success, colored by booster version

Why it matters

- The dashboard lets a non-technical bidder test payload and site combinations directly, without rerunning any of the notebooks.



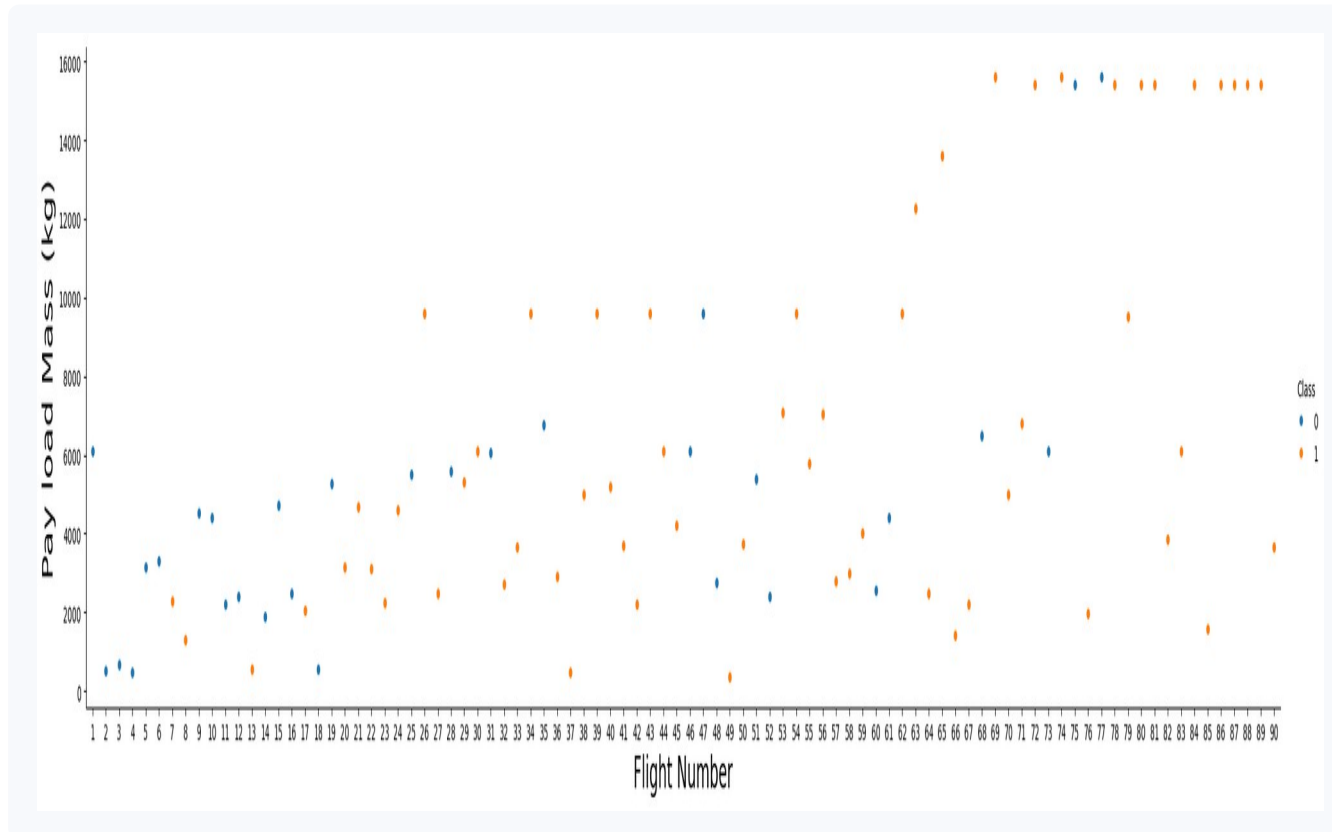
Notebook / App Code: github.com/ptah222/IBM-Data-Science-Capstone-Project



EDA RESULTS

Flight Number & Payload vs. Launch Outcome

Each point is one Falcon 9 launch; orange = successful landing (Class 1), blue = failure (Class 0)



Flight Number vs. Payload Mass, colored by outcome (Class)



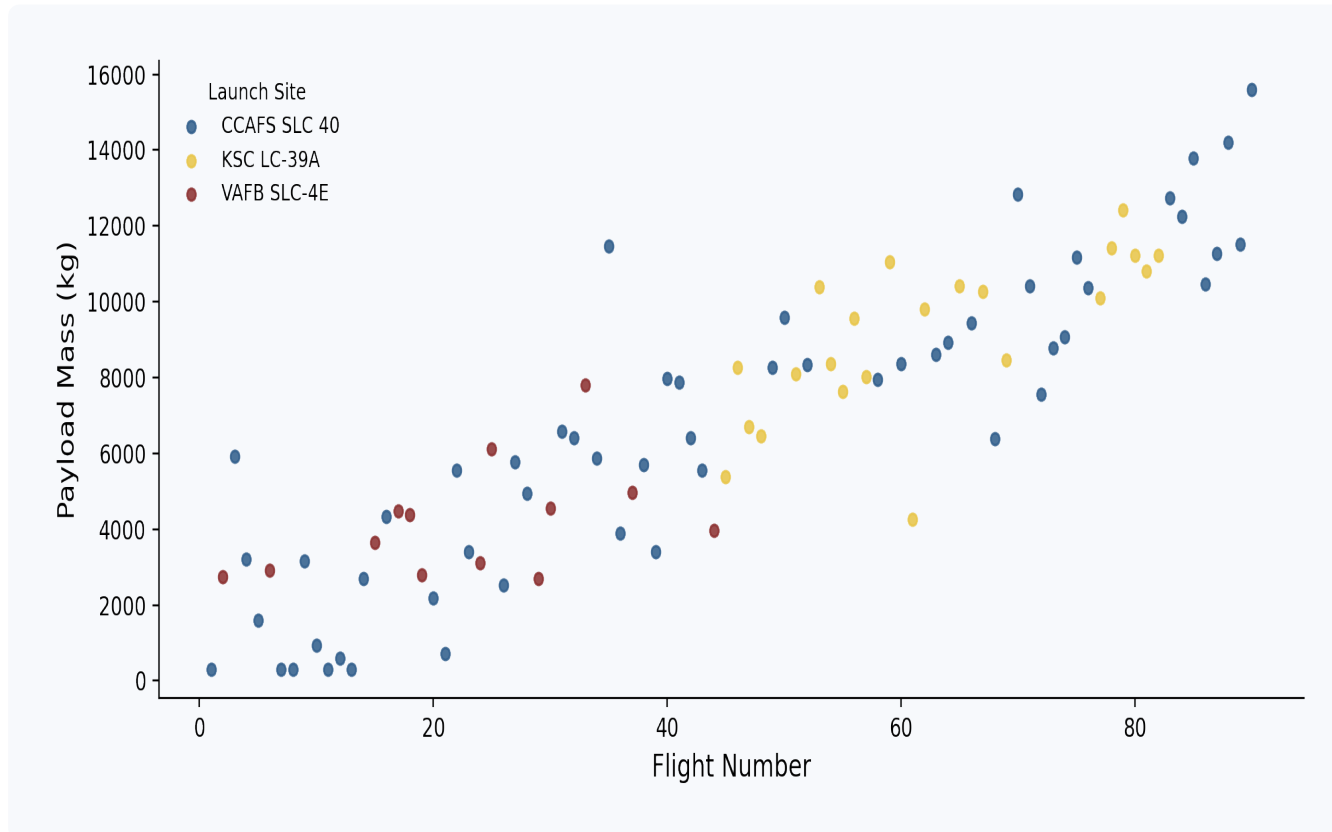
EDA Notebook: github.com/ptah222/IBM-Data-Science-Capstone-Project



EDA RESULTS

Flight Number vs. Launch Site

Each point is one Falcon 9 launch, colored by launch site; payload mass grows with flight number as SpaceX matured



Flight Number vs. Payload Mass, colored by launch site



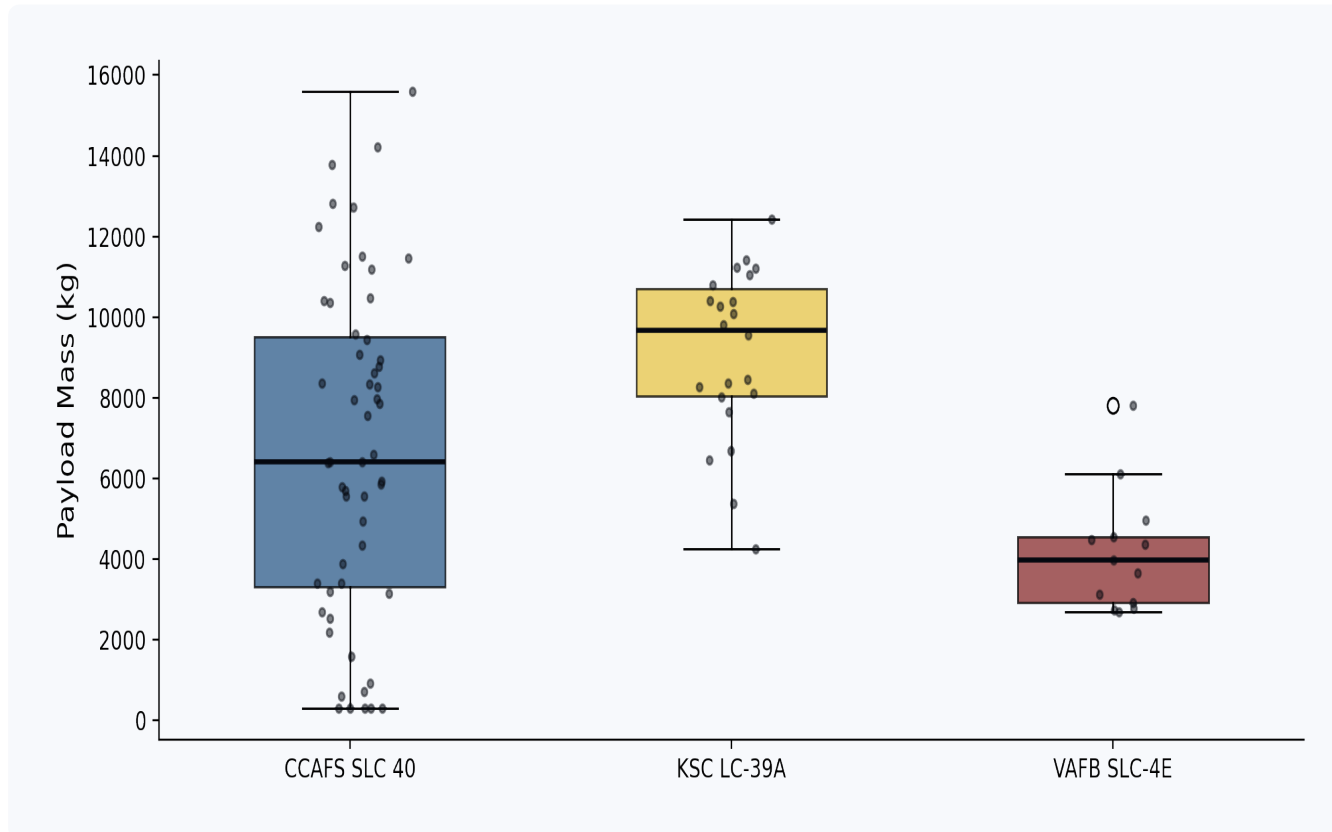
EDA Notebook: github.com/ptah222/IBM-Data-Science-Capstone-Project



EDA RESULTS

Payload Mass vs. Launch Site

Distribution of payload mass carried from each pad; KSC LC-39A skews heaviest, VAFB SLC-4E lightest



Payload Mass by Launch Site (box plot with individual launches overlaid)

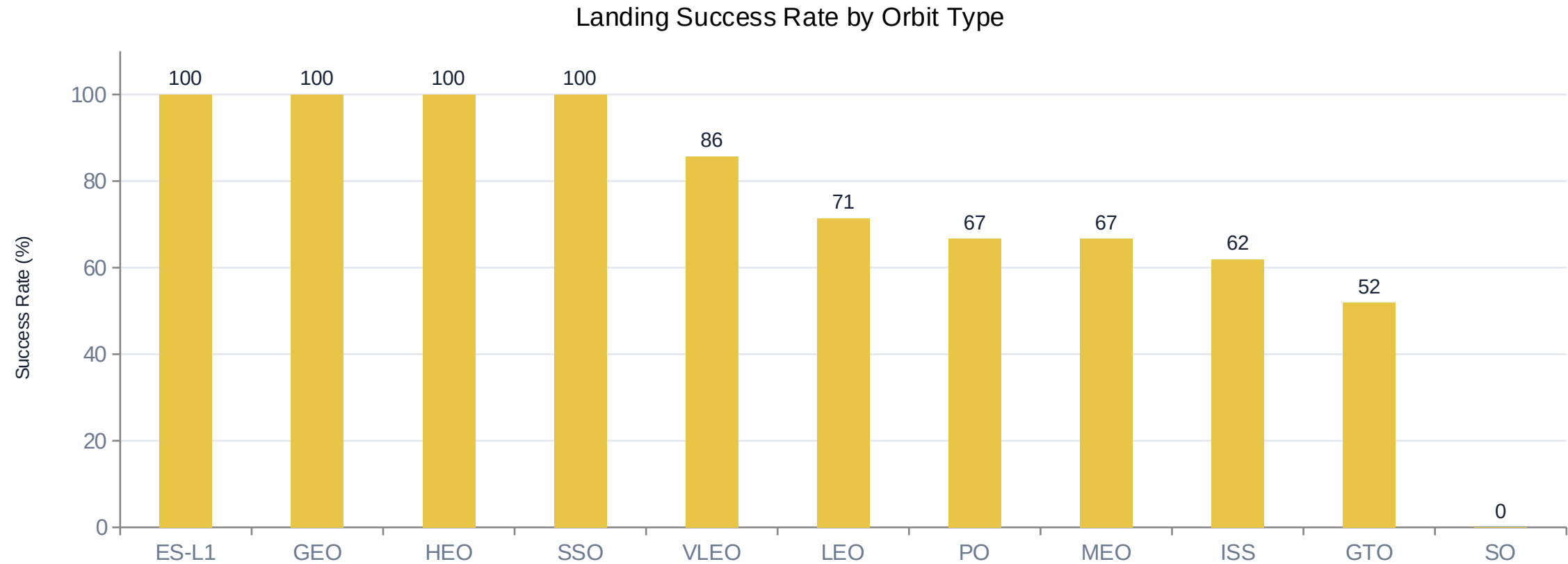


EDA Notebook: github.com/ptah222/IBM-Data-Science-Capstone-Project



Success Rate by Orbit Type

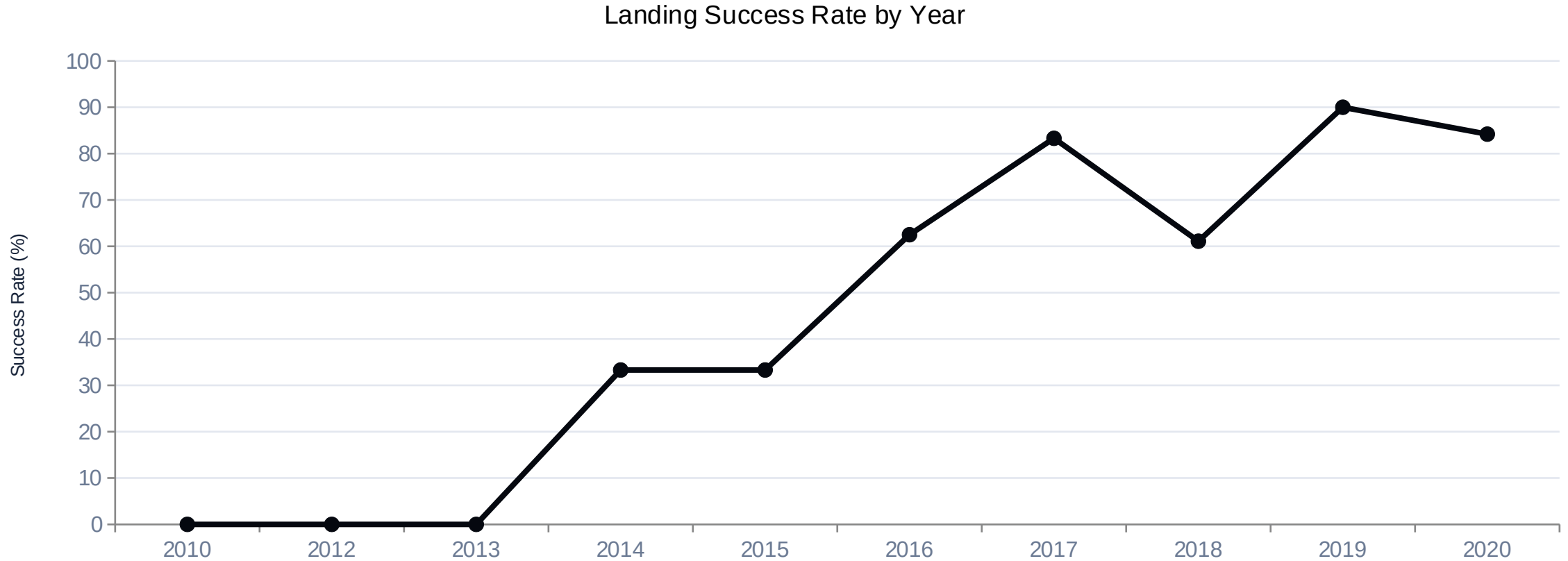
ES-L1, GEO, HEO and SSO orbits show a 100% landing success rate; GTO is lowest at 51.9%, reflecting its higher energy requirements.





Yearly Launch Success Trend

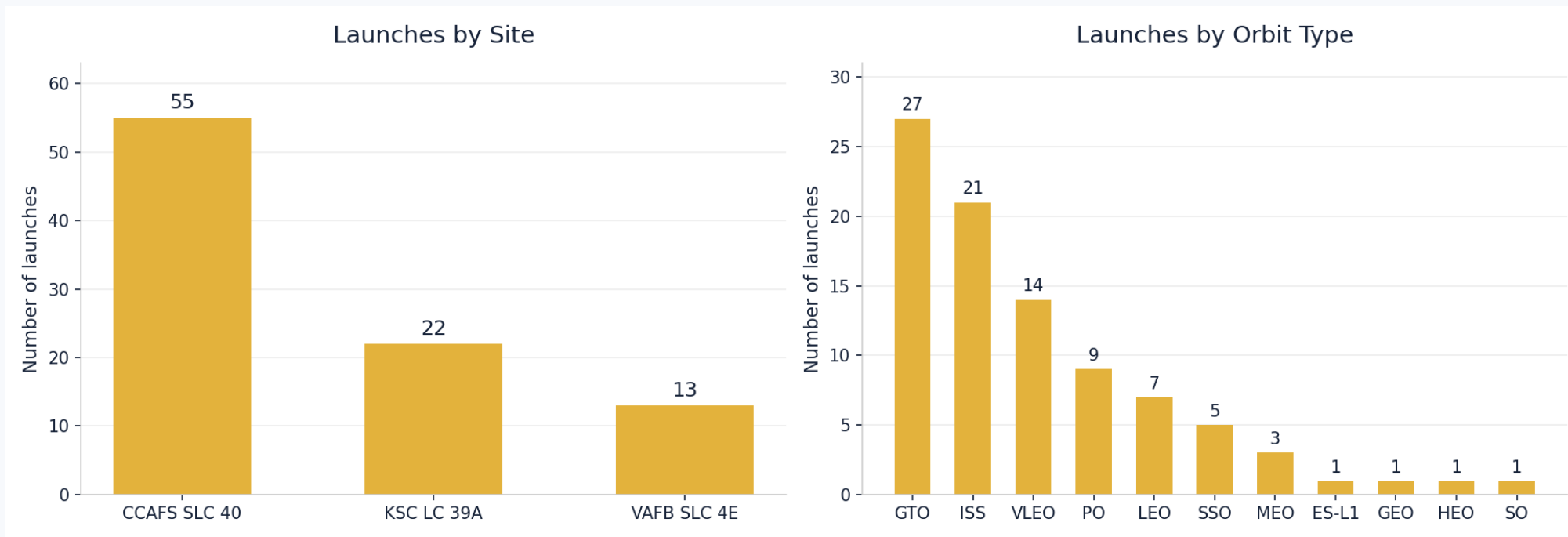
Success rate climbed from 0% before 2014 to 90% in 2019 as SpaceX iterated on Block 5 boosters and landing technique.





Launch Site & Orbit Distribution

Counts computed with `value_counts()` over the 90-launch wrangled dataset



- CCAFS SLC 40 handled 55 of 90 launches; GTO and ISS together account for 48, so the dataset is dominated by one pad and two orbit classes — worth remembering when reading per-site and per-orbit success rates.

Both charts total 90 launches



EDA Notebook: github.com/ptah222/IBM-Data-Science-Capstone-Project



Launch Sites & Payload Totals

Query	Sample Result
Distinct launch site names	CCAFS LC-40, CCAFS SLC-40, KSC LC-39A, VAFB SLC-4E
Total payload mass carried for customer NASA (CRS)	45,596 kg

What these queries establish

- The SQL table lists four pad identifiers, but CCAFS LC-40 and CCAFS SLC-40 are the same pad renamed in 2016 — the wrangled dataset collapses them to three sites.
- NASA (CRS) resupply missions account for 45,596 kg of payload in total, making NASA the single largest customer by mass.
- The early F9 v1.1 booster averaged only 2,928.4 kg per launch — later Block 5 boosters carry several times that, so payload mass and booster generation move together.

Results copied from the executed EDA-SQL notebook (SpaceX.csv loaded into SPACEXTABLE).



Success Rates & Rankings

Query	Sample Result
First successful ground-pad landing date	2015-12-22
Boosters: drone-ship success & payload 4,000–6,000 kg	F9 FT B1022, F9 FT B1026, F9 FT B1021.2, F9 FT B1031.2
Mission outcome counts (all records)	Success: 100 Failure (in flight): 1
Boosters carrying the maximum payload mass	12 F9 B5 boosters — B1048.4, B1049.4, B1051.3, B1056.4, ...

Results copied from the executed EDA-SQL notebook (Spacex.csv loaded into SPACEXTABLE).



Time Analysis

Query	Sample Result
2015 drone-ship landing failures, by month	January and April (F9 v1.1 B1012 and B1015, both CCAFS LC-40)
Ranked landing outcomes, 2010-06-04 to 2017-03-20	1) No attempt — 10 2) Success drone ship — 5 3) Failure drone ship — 5 4) Success ground pad — 3

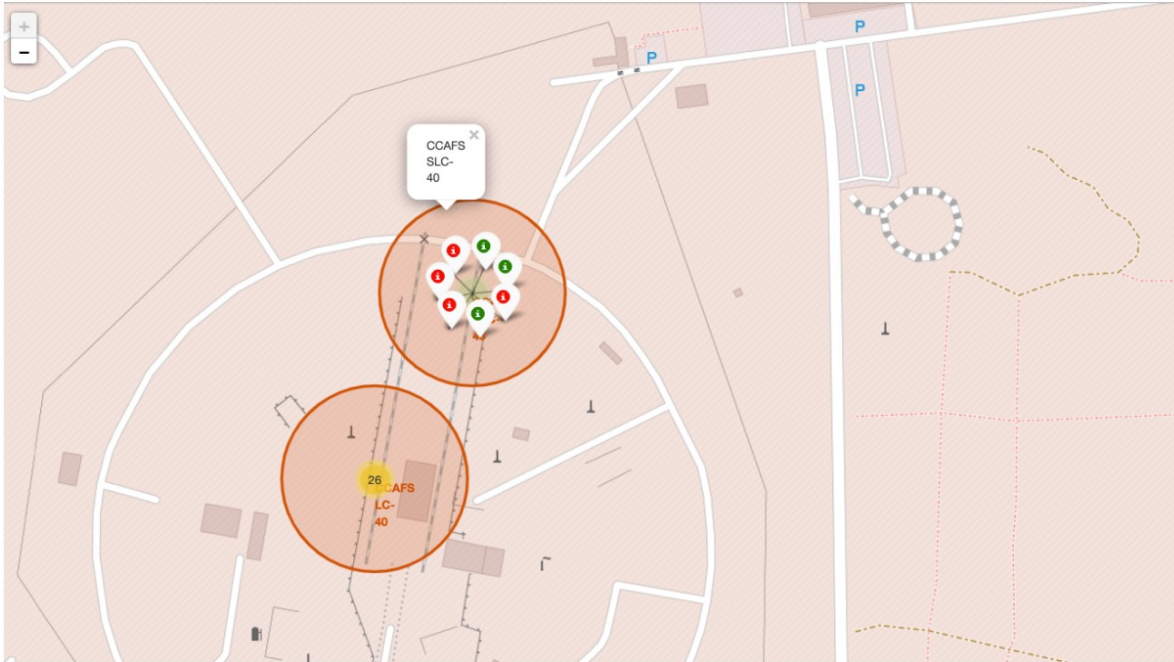
What the time analysis shows

- Both 2015 drone-ship failures came in the first half of the year, on F9 v1.1 boosters flying from CCAFS LC-40 — the period when barge landing was still unproven.
- Through March 2017 the most common outcome was still no recovery attempt (10), with drone-ship attempts split evenly at 5 successes and 5 failures.
- Ground-pad returns were rarer but more reliable (3 successes, no failures), which is why SpaceX kept investing in both recovery modes.

Results copied from the executed EDA-SQL notebook (SpaceX.csv loaded into SPACEXTABLE).



Folium Map — Launch Site Markers



From the color-labeled markers in marker clusters, you should be able to easily identify which launch sites have relatively high success rates.

Site markers — folium.Circle + folium.Marker

```
nasa_coordinate = [29.5597, -95.0831]
site_map = folium.Map(
    location=nasa_coordinate, zoom_start=5)

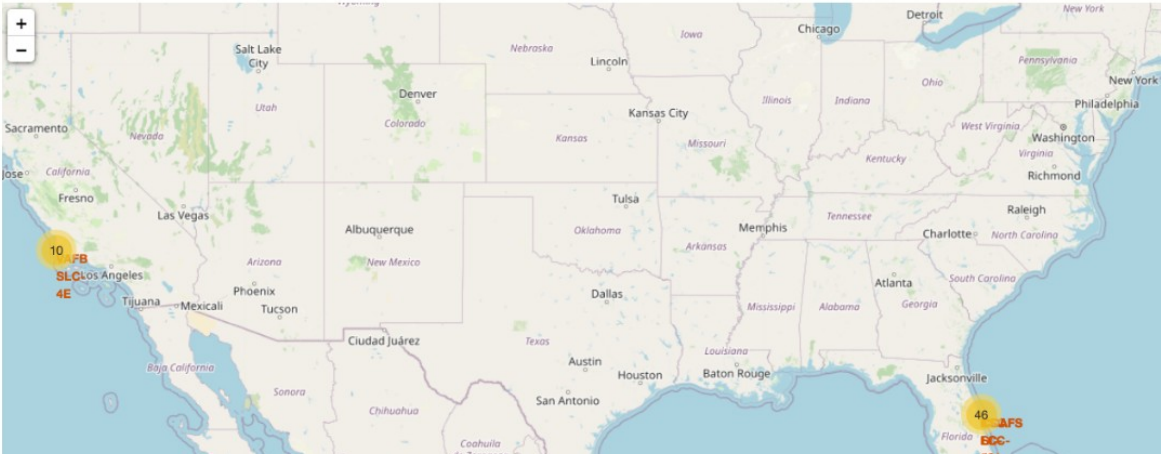
for _, row in launch_sites.iterrows():
    coord = [row['Latitude'], row['Longitude']]
    folium.Circle(
        coord, radius=1000, color='#000000',
        fill=True
    ).add_child(
        folium.Popup(row['LaunchSite'])
    ).add_to(site_map)
    folium.Marker(
        coord, popup=row['LaunchSite']
    ).add_to(site_map)
```



Folium Notebook: github.com/ptah222/IBM-Data-Science-Capstone-Project



Folium Map — Launch Records & Proximities



All 90 launch records, color-coded by outcome and clustered by site

Marker clusters & proximity analysis

- Every launch record is added to a MarkerCluster, with `marker_color` set from the Class label — green for a successful landing, red for a failure.
- Clusters expand on zoom, so overlapping launches at CCAFS SLC-40, KSC LC-39A and VAFB SLC-4E stay readable.
- `folium.PolyLine` plus a Haversine distance calculation measure each site to its nearest coastline, railway, highway and city.

What the map shows

- All three sites sit within roughly 1 km of the coast — failed stages fall over water — while cities are kept much further away.
- CCAFS SLC-40 and KSC LC-39A carry the bulk of launches and show visibly denser green clusters than VAFB SLC-4E.

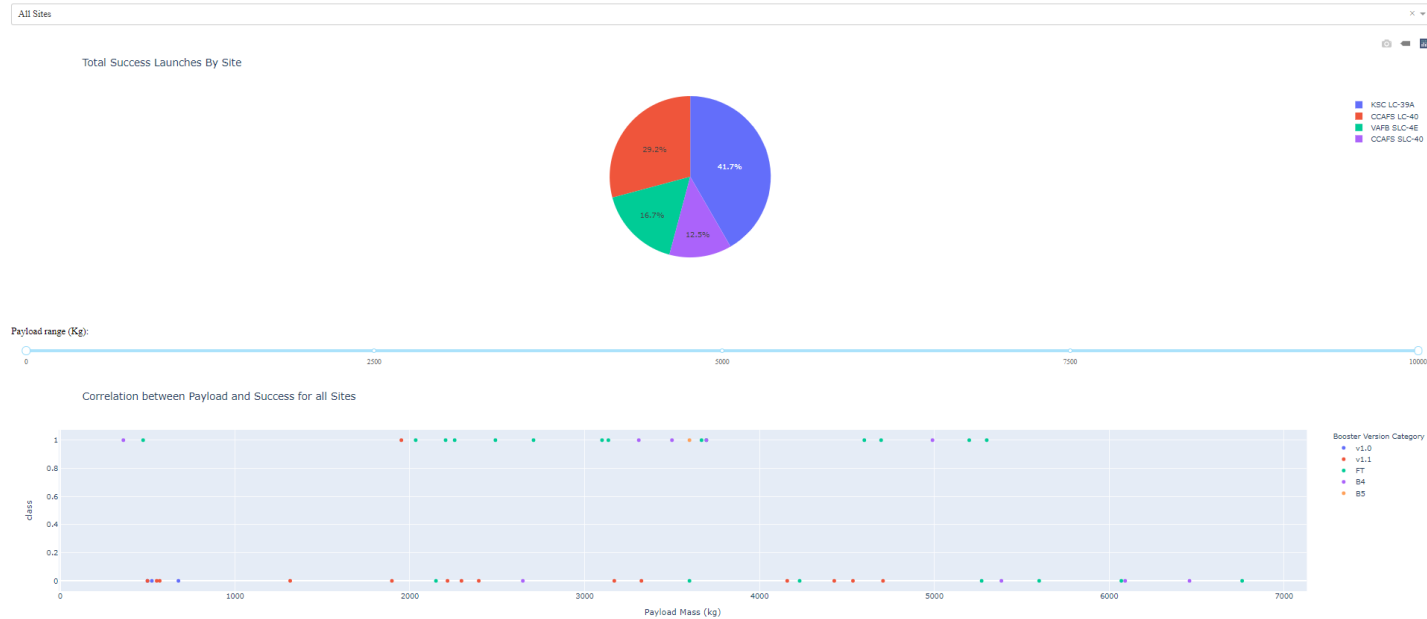


Folium Notebook: github.com/ptah222/IBM-Data-Science-Capstone-Project



Plotly Dash — Interactive Dashboard

SpaceX Launch Records Dashboard



Success pie chart

- With All Sites selected, the pie splits total successful launches by site: KSC LC-39A leads at 41.7%, followed by CCAFS LC-40 at 29.2%.
- Selecting a single site switches the chart to that site's own success vs. failure share.

Payload vs. outcome scatter

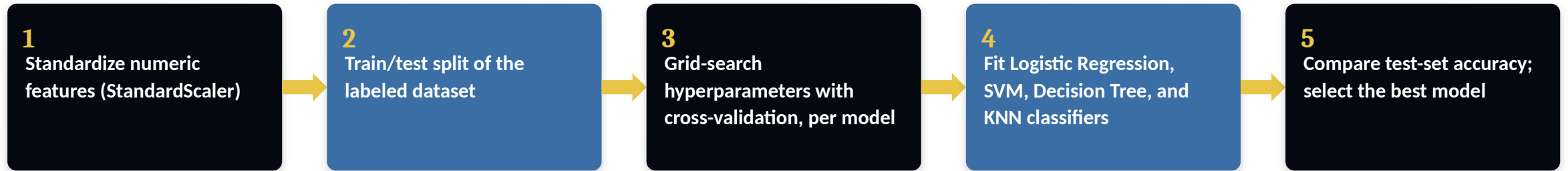
- Points are colored by booster version category, and the payload range slider filters the axis.
- Successful landings concentrate below roughly 6,000 kg; FT and B4 boosters account for most of the successes.



Dash App Code: github.com/ptah222/IBM-Data-Science-Capstone-Project



Predictive Analysis Methodology



- Target variable: Class (1 = successful landing, 0 = failure), engineered during data wrangling
- Features: flight number, payload mass, orbit type, launch site, booster version/grid fins/legs, and reused-booster flag
- Each model tuned via GridSearchCV; best estimator per model type evaluated on the held-out test set

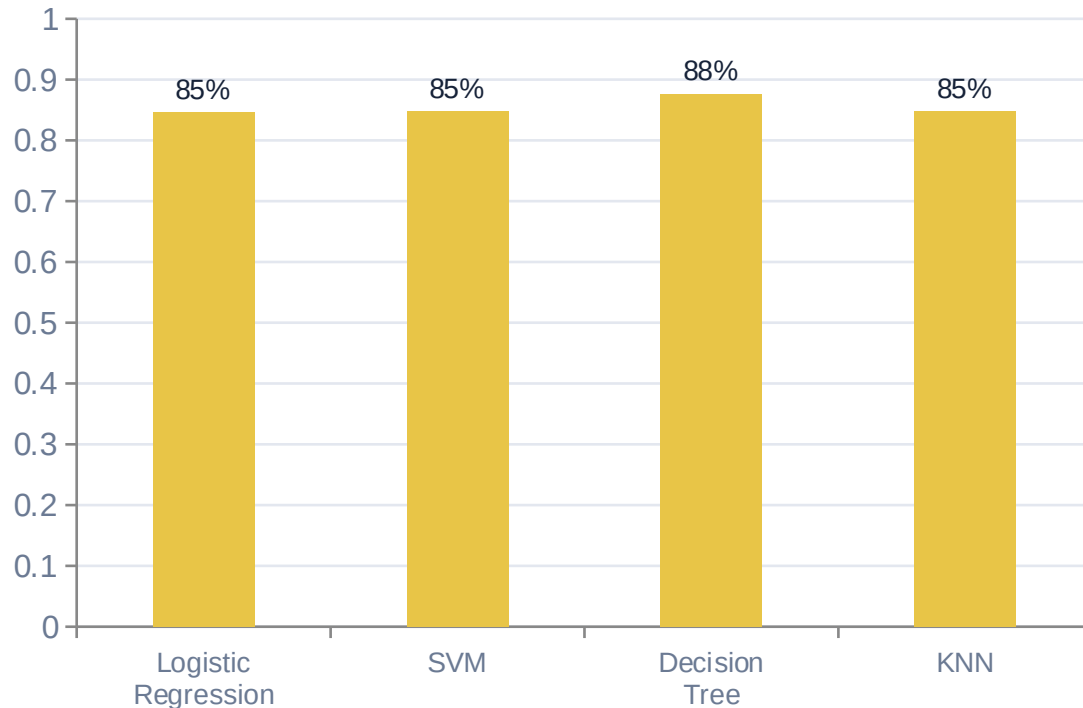


Modeling Notebook: github.com/ptah222/IBM-Data-Science-Capstone-Project



Model Comparison & Best-Model Results

Model Accuracy Comparison



Best Model: Decision Tree — 87.7% Accuracy

Best params: gini, max_depth=8, max_features=sqrt, min_samples_leaf=4, min_samples_split=2

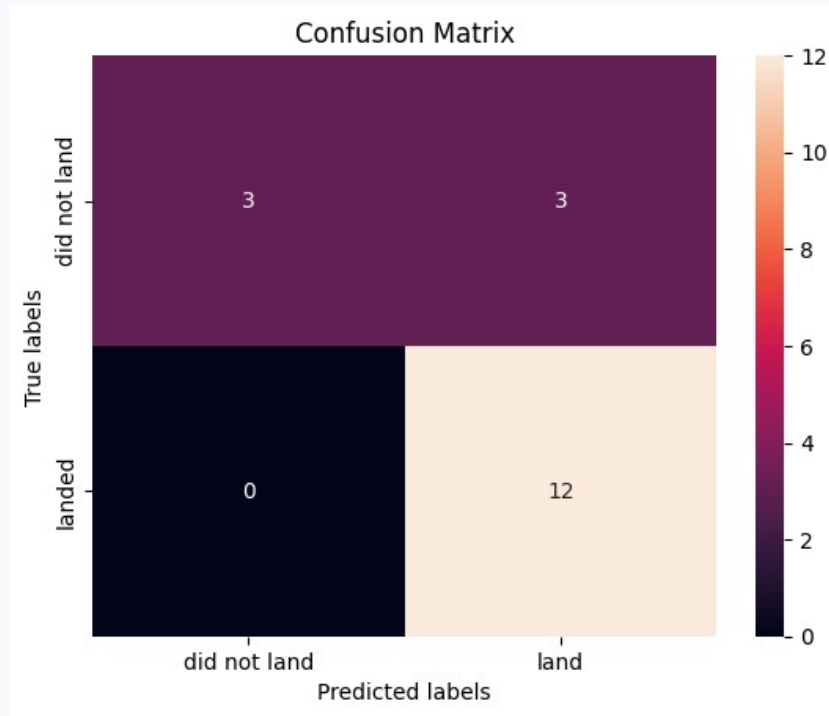
- Tuned with GridSearchCV over a 5-fold cross-validation split.
- All four models exceed 84%, so the signal is in the features, not the algorithm.
- All four models produce an identical confusion matrix and the same 83.3% test accuracy — the CV score is what separates them.
- Why the tree wins: it splits on threshold-style rules (e.g. payload > 7,000 kg, orbit = GTO) that mirror the real physics of a landing attempt, so it captures non-linear cutoffs the linear and distance-based models can't.

Decision Tree scored best in cross-validation (87.7%), ahead of SVM/KNN (84.8%) and Logistic Regression (84.6%), and was selected as the final model.



Confusion Matrix — Decision Tree

Held-out test set of 18 launches; identical for all four classifiers



Rows = true outcome, columns = predicted outcome

Reading the matrix

- True positives: 12 — every launch that actually landed was predicted to land.
- False negatives: 0 — the model never misses a successful landing.
- True negatives: 3, false positives: 3 — of six real failures, half were wrongly predicted as successes.
- Accuracy = $15/18 = 83.3\%$ on the test set.

Why it matters

- The single weakness is false positives. For a competitor bidding against SpaceX, that means the model may assume a cheap reusable launch when the stage would in fact be lost — the costly direction to be wrong in.



Modeling Notebook: github.com/ptah222/IBM-Data-Science-Capstone-Project



Conclusion

- The Decision Tree Classifier was the best performing model, at 87.7% cross-validated accuracy — ahead of SVM and KNN (84.8%) and Logistic Regression (84.6%).
- The strongest predictors of a successful landing are flight number (accumulated experience), orbit type, launch site and payload mass.
- Landing success is strongly time-dependent: 0% before 2014, rising to 90% by 2019 as SpaceX iterated toward the Block 5 booster.
- Orbit type matters sharply — ES-L1, GEO, HEO and SSO reached 100% success, while high-energy GTO missions managed only 51.9%.
- KSC LC-39A had the highest per-site success rate; heavier payloads above ~7,000 kg showed noticeably fewer successful landings.
- A competitor can therefore estimate whether SpaceX will recover a first stage — and so whether a bid is priced against a ~\$62M reusable launch or a full-cost expendable one.
- **Innovation angle: the model's predicted landing probability could power a live "reusability index" that re-prices each bid mission-by-mission — instead of one flat rate — turning a static cost estimate into a dynamic pricing tool.**



APPENDIX

GitHub Repository Links



Data Collection — API notebook: <https://github.com/ptah222/IBM-Data-Science-Capstone-Project/blob/0584d6d27082113542dc9254eb112fbda10bb4f8/part%201%20jupyter-labs-spacex-data-collection-api.ipynb>



Data Collection — Web Scraping notebook: <https://github.com/ptah222/IBM-Data-Science-Capstone-Project/blob/bb0d5b67337cbdf7ec640d6aaa9d6e046e613737/part%201%20jupyter-labs-webscraping.ipynb>



Data Wrangling notebook: <https://github.com/ptah222/IBM-Data-Science-Capstone-Project/blob/03cf6dd686448d61155a02de20f39b63c7e72e58/labs-jupyter-spacex-Data%20wrangling.ipynb>



EDA — Data Visualization notebook: <https://github.com/ptah222/IBM-Data-Science-Capstone-Project/blob/f95268ea8e1c61bacbb2be381b085205bc301cb0/edadataviz.ipynb>



EDA — SQL notebook: https://github.com/ptah222/IBM-Data-Science-Capstone-Project/blob/f95268ea8e1c61bacbb2be381b085205bc301cb0/part%202%20jupyter-labs-eda-sql-coursera_sqllite.ipynb



Interactive Visual Analytics — Folium notebook: https://github.com/ptah222/IBM-Data-Science-Capstone-Project/blob/c4cfb8f28e87c02f9103472e11f476a3fd3c9572/lab_jupyter_launch_site_location_folium.ipynb



Interactive Visual Analytics — Plotly Dash app: <https://github.com/ptah222/IBM-Data-Science-Capstone-Project/blob/0a065d79f08a9dac8a9bd57f4a0756986d7ecb52/spacex-dash-app.py>



Predictive Analysis — Modeling notebook: https://github.com/ptah222/IBM-Data-Science-Capstone-Project/blob/2f4e7255690fa8ce7c7aa9f9847a3803ac14a9fd/SpaceX_Machine%20Learning%20Prediction_Part_5.ipynb